

Created by the Aerospace Cyber Defence Centre of Excellence (CEC)

Documentation

WAC (Windows Artefacts Collector)

Windows artefact collection software
for digital forensic investigation



SYSTEM
ARTEFACTS



USER
FILES



DIGITAL
EVIDENCE

A TOOL IN SUPPORT
OF DIGITAL
INVESTIGATION

WAC — Windows Artefacts Collector

[User manual](#)

Software version 1.3.1

Document edition September 2026

Publisher Aerospace Cyber Defence Centre of Excellence (CEC)
French Air and Space Force Academy, Air Base 701, Salon-de-Provence

This document describes how to use WAC: preparing and running a collection, reading and verifying its results, and the traces it leaves on the examined machine. The figures quoted were measured on a Windows 11 test virtual machine; they are orders of magnitude, not commitments.

Contents

1 Overview	2
1.1 What WAC is for	2
1.2 The guiding principle: a minimal footprint	2
1.3 What WAC does not do	2
2 How it works	3
2.1 Reading the disk directly	3
2.2 Exhibit store and working copy	3
2.3 Registry hives and transaction logs	4
2.4 Two-pass extraction	4
2.5 What is still read live	4
3 Preparing a collection	5
3.1 Requirements	5
3.2 The collection medium	5
3.3 Before going on site	6
4 Running a collection	7
4.1 Command line	7
4.2 Options	7
4.3 Examples	7
4.4 The --binary option	8
4.5 What the screen shows	9
4.6 Duration	10
5 The results	11
5.1 Layout	11
5.2 The exhibit store manifest	11
5.2.1 Header	11
5.2.2 Each exhibit	11
5.3 Verifying a collection	12

5.3.1	The seal	12
5.3.2	Each exhibit	13
5.4	The investigation log	13
5.5	The artefact files	14
5.5.1	System and accounts	14
5.5.2	Program execution	14
5.5.3	Files and browsing	15
5.5.4	Devices	15
5.5.5	Event logs (--events)	16
6	Reading the data	17
6.1	Timestamps	17
6.2	A missing field is not an empty field	17
6.3	Artefact not collected	17
6.4	Undecoded data	18
6.5	Paths	18
7	Traces left on the examined machine	19
7.1	What WAC still does	19
7.2	What WAC no longer does	19
7.3	What no tool can avoid	20
8	Known limitations	21
9	Troubleshooting	22
A	Glossary	23

Overview

1.1 What WAC is for

WAC collects, from a running Windows computer, the artefacts a digital investigation needs: registry, event logs, Prefetch files, shortcuts and jump lists, scheduled tasks, services, accounts, processes and sessions. It delivers them as **JSON** files, ready to be analysed or fed into a processing pipeline.

It runs from a USB stick, without installation, and writes everything it produces to that stick.

1.2 The guiding principle: a minimal footprint

Examining a machine changes it. Every tool leaves traces: opened files whose last-access date changes, services that write to the logs when solicited, registry keys read through the API. These traces mix with the ones being looked for.

WAC is designed so that what it leaves behind is as close to nothing as the task allows, and so that **whatever it cannot avoid is written down**. In practice:

- the disk is read **directly**, read-only, without the system opening any file: no access date is modified;
- no key of the machine's registry is opened: the hives are copied, then read on the copy;
- no Volume Shadow Copy, no COM or WMI call, no logging service is solicited;
- nothing is written to the examined disk;
- every operation is recorded, with the trace it is expected to leave, in `investigation.json`.

1.3 What WAC does not do

- WAC **does not analyse**: it collects and formats. Interpretation is the analyst's.
- It does not make a full disk image: it extracts the files that hold artefacts.
- It does not capture memory. The only volatile data collected are the process list, the open sessions and the state of the services.

How it works

2.1 Reading the disk directly

WAC opens the volume (\\.\C:) read-only, walks the NTFS file table (\$MFT) itself and extracts the content of the files it needs. The system sees no file being opened: no access date updated, no directory walked, no lock.

This reading handles the NTFS features Windows normally hides:

NTFS compression

Windows 11 compresses, among others, the event log folder. WAC decompresses it itself.

Compact OS (WOF)

The system binaries of Windows 10 and 11 are stored compressed in a hidden stream; seen through the API, they look ordinary. WAC decompresses them (XPRESS variants).

Split attributes

A heavily fragmented file has its information spread over several records; WAC gathers them.

Preallocated files

Beyond the valid data length, NTFS returns zeros, even if the disk still holds remnants of former files. WAC does the same: the extracted content is the one any ordinary acquisition would see, and so is its fingerprint.

2.2 Exhibit store and working copy

A digital exhibit is never examined on itself: a copy is taken and sealed, and the work is done on a second copy. If the analysis damages something, the sealed copy remains and the operation can be redone.

WAC applies this procedure to **every** extracted file, including those it does not modify. The two directories keep their French names in the output:

exhibits/ — the exhibit store

the raw copies, as read from the volume. This directory is never reopened for writing after extraction. It holds the manifest identifying each exhibit, and the manifest's seal.

working/ — the working copy

copied from the exhibit store and verified by fingerprint. The only modifications (replay of the hives' transaction logs) are made here, and every collector reads from here.

Keep in mind

Every extracted file is therefore written **twice** to the collection medium. Plan for the space (see section 3.2).

2.3 Registry hives and transaction logs

A hive copied from a running machine lacks its latest changes: they are still in its transaction logs (.LOG1, .LOG2). WAC extracts these logs and **replays them onto the working copy**, so that the analysis bears on the machine's actual state.

Everything WAC writes onto a copy can be undone: before the replay, the original content of every replaced page is saved in an undo journal (<hive>.undo). The raw copy stays intact in the exhibit store.

2.4 Two-pass extraction

Where each user's hives live is recorded in the SOFTWARE hive. WAC therefore first extracts the machine's hives (SYSTEM, SOFTWARE, SAM, Amcache), reads the profile list in the copy of SOFTWARE, then extracts each profile's hives (ntuser.dat, usrClass.dat).

Nothing is assumed about C:: the system drive is detected, and a profile on another disk (for instance D:\Users\...) is read on its own volume.

2.5 What is still read live

Three pieces of information describe the very moment of the collection and are held in no file. They are read live, with a single query each:

- the running **processes** (a kernel snapshot);
- the open **sessions**;
- the **current state of the services** (their configuration is read from the copied hive).

Preparing a collection

3.1 Requirements

- Windows 10 or later, 64-bit. WAC checks it at start-up.
- **Administrator rights:** reading a volume directly requires them. WAC checks this too.
- **NTFS** volumes. A file on another file system (CD-ROM, FAT stick) cannot be read directly; it is reported as unread, never opened another way.

3.2 The collection medium

WAC runs from the collection medium (usually a USB stick) and writes all its results there. Choose an **empty** medium, preferably USB 3: writing is the most expensive part of a collection.

The space needed depends on the options. Orders of magnitude measured on an ordinary Windows 11 installation, for the raw copies alone:

Content	Size of the raw copies
Hives, Prefetch, shortcuts, jump lists (always)	about 200 to 350 MB
Event logs and their message binaries (--events)	about 250 MB
Referenced binaries not authenticated as Microsoft's (--binary)	a few hundred MB

Since every copy is written twice (exhibit store and working copy), **double these figures**. A full collection (--events --binary) thus takes about 1.5 to 2 GB on an ordinary installation.

WAC checks the location **before its very first write** and refuses to start:

- if the working directory already holds files — an earlier collection would be analysed instead of this one;
- if the free space is clearly insufficient.

If space runs out while binaries are being collected, they are hashed without being copied, and the investigation log says so.

3.3 Before going on site

- Use a **new output folder** for each machine (option `--output`).
- Note the time of the intervention and the operator's identity: WAC records them too, but an independent record is useful.
- Keep in mind that plugging the stick in and running WAC leave unavoidable traces (chapter 7): they are known and documented, so that they are not mistaken for the activity under study.

Running a collection

4.1 Command line

Open a command prompt **as administrator**, move to the collection medium, then run:

```
WAC.exe [--events] [--binary] [--dump] [--output=folder] [--loglevel=N] [--debug]
```

Every option is optional and disabled by default.

4.2 Options

Option	Effect
--events	Extracts and parses the event logs (.evt _x), and rebuilds each event's readable message from its provider's resources. Adds about 250 MB of copies.
--binary	Fingerprints (MD5, SHA-1, SHA-256) every file referenced by an artefact, by reading the disk directly. Referenced executables, libraries, drivers and scripts are also collected into the exhibit store. See section 4.4.
--dump	Adds the raw hexadecimal value of shellbag and .lnk shortcut items, for later decoding.
--output=folder	Output folder, relative to the current directory. Default: output.
--loglevel=N	Enables a detailed text log in the current directory, named after the program run (WAC.exe.log for WAC.exe): 0 none, 1 per artefact type, 2 per artefact, 3 per sub-function (debugging).
--debug	Prints the NTFS reader's trace on standard error (path resolution, index blocks, cluster runs). For diagnosis only.
--help, /?	Shows the help.

4.3 Examples

Standard collection:

```
E:\> WAC.exe --output=accounting-pc-01
```

Full collection, with logs, binaries and a detailed log:

```
E:\> WAC.exe --events --binary --output=accounting-pc-01 --loglevel=2
```

4.4 The --binary option

A fingerprint is enough to query a public database (VirusTotal, for instance) **without sending it the file**. But it says nothing about a binary nobody knows — precisely the one that matters to the investigation — and a binary left behind may be gone by the time a detection comes in. This is why --binary also copies potential payloads:

- **collected:** exe, dll, sys, ocx, cpl, scr, drv, efi, com, msi and the scripts ps1, psm1, bat, cmd, vbs, vbe, js, jse, wsf, wsh, hta;
- **collected as well:** Office documents **able to carry macros**, an intrusion vector as common as an executable — legacy binary formats (doc, xls, ppt, pps, dot, xlt, pot), “m” formats (docm, dotm, xlsx, xlsm, pptm, potm, ppsm), xlsb, add-ins (xla, xlam, xll, wll, ppa, ppam), Publisher (pub), Visio (vsd, vsdm, vstm, vssm) and Access (mdb, accdb, accde). They appear in the traces as targets of shortcuts or jump lists, or among the files loaded by Word or Excel in their Prefetch;
- **only hashed:** other documents and data files, including docx, xlsx and pptx, which cannot contain macros. Copying them would turn the collection into a copy of the user’s files.

The referenced files come from processes, services (binary and DLL), scheduled tasks, files loaded by programs (Prefetch), Shimcache, Amcache, and targets of shortcuts and jump lists. Each file is read only once, however many artefacts cite it, and **identical content is stored only once**: the other paths appear in the manifest as exhibits sharing that content (SharedExhibit).

Authentic Microsoft binaries: hashed, not collected

Most referenced binaries are Windows components, identical on every machine of the same build. Their origin can be proven on the machine itself, with no list to carry: Windows’ **catalogs** (System32\CatRoot), signed by Microsoft, list the fingerprint of every system file; individually signed binaries (Edge, OneDrive, Office) carry an **embedded signature**. A binary authenticated this way is hashed without being collected; every other one is collected — including a system binary replaced by an attacker, whose fingerprint no longer matches.

This check leaves **no trace**: catalogs and binaries are read directly from the disk, and the cryptographic verification is done in memory by WAC, up to Microsoft root certificates embedded in the executable. No service is solicited, the machine’s certificate store is not consulted, and revocation is not checked online.

Only signers Microsoft uses for its own code are accepted. **Third-party** drivers certified by Microsoft (signers “Hardware Compatibility Publisher” and “Early Launch Anti-malware Publisher”) are still collected: a vulnerable but signed driver is a classic attack path.

The catalogs that justified not collecting a file go into the exhibit store, so that a third party can redo the check. In the artefact files, the Signature field (or ServiceDllSignature, SignatureTarget...) states the authentication used, for instance "Microsoft (catalogue Package_xxx.cat)" or "Microsoft (signature intégrée)" (embedded signature).

Scripts are authenticated the same way: by Windows' catalogs and, for PowerShell (ps1, psm1, psd1, ps1xml...), by their embedded signature block (# SIG # Begin signature block). A Windows Script Host script (vbs, js, wsf) signed only within its text, and not by a catalog, is still collected.

On the test machine, this check brings collection down from 2,131 binaries (3.3 GB) to 92 (335 MB), without lengthening the collection; the scripts still collected are the unsigned ones.

Since reading is direct, collecting leaves no more trace than hashing — it only takes space on the medium.

4.5 What the screen shows

WAC displays each step, followed by OK or an error message. A failed step does not stop the collection: whatever is accessible is gathered, and whatever is missing is recorded.

```
[PREREQUISITE VERIFICATION]
- Check OS >= Windows 10 : OK
- Check administrator rights : OK
- Checking the collection medium : OK
[SEARCHING FOR ARTIFACTS IN WINDOWS API]
- Extraction of SESSIONS: OK
- Extraction of PROCESS: OK
[RAW EXTRACTION]
- Extracting system hives (raw NTFS) : OK
- loading the HKLM\SYSTEM key : OK
- loading the HKLM\SOFTWARE key : OK
- Listing USER PROFILES (SOFTWARE hive) : OK
- Extracting user hives (raw NTFS) : OK
- Extracting file artefacts (raw NTFS) : OK
[SEARCHING FOR ARTIFACTS IN THE REGISTRY]
- Extracting USERASSIST Registry Keys : OK
...
[SEARCHING FOR ARTIFACTS IN FILES]
- Extracting PREFETCHS : OK
...
[SEARCHING FOR ARTIFACTS IN EVENT LOGS]
- Extraction of EVENTS: OK
[EXHIBIT STORE]
- Copying collected binaries to the working directory : OK
- Sealing the exhibit store (manifest + SHA-256) : OK
[INVESTIGATION LOG]
- Writing investigation.json : OK
END, Time elapsed : 309 s
```

Sealing the exhibit store is the **last** operation on it: every exhibit, including those added

while the event logs are processed, is covered by the manifest.

4.6 Duration

The duration depends mostly on the hardware (USB 2 or 3, disk, processor) and the options. On the test virtual machine (NVMe disk): about 70 s with `--events`, about 310 s with `--events` and `--binary`. On a USB stick, writing dominates.

The results

5.1 Layout

```
<output>/
  exhibits/           exhibit store: raw copies, never rewritten
    MANIFEST.json     identifies each exhibit and the collection
    MANIFEST.sha256   the manifest's fingerprint: its seal
    Windows/...       the exhibits, under their original paths
    Users/...
  working/           working copies (replayed hives)
  investigation.json  the collection log
  *.json             the artefacts
```

A file from a volume other than the system one is stored under `_volume_D\...`; the JSON always gives the original path, with its real drive letter.

5.2 The exhibit store manifest

`exhibits/MANIFEST.json` identifies the collection and each exhibit.

5.2.1 Header

Tool

tool name, command line, build date.

Host

machine name, system drive, time zone of the suspect (read from its SYSTEM hive) and of the collecting machine.

Operator

account that ran the collection, and whether it was elevated.

Custody

procedure statement, start of extraction, time the manifest was written, number of exhibits and failures, total size, hash algorithms, volumes read (letter, serial number, file system).

5.2.2 Each exhibit

Field	Meaning
SourcePath	Original path, with drive letter.
ExhibitPath	Location of the raw copy, relative to the output folder.
Method	Collection method (direct NTFS reading, volume, purpose).
Result	OK, or the error code and message. Failures are listed in the manifest: a missing exhibit would read as one never looked for.
MD5, SHA1, SHA256	The three fingerprints, computed while writing: they bear on what was read from the volume.
Bytes	Extracted size.
DeclaredBytes	Size declared by NTFS, present only when it differs from the extracted size (truncated extraction).
ValidDataBytes	Valid data length of a preallocated file (event logs): beyond it, the exhibit holds zeros by definition.
SharedExhibit	true when the content, identical byte for byte, is already stored under another exhibit: ExhibitPath points to it. The source remains an exhibit in its own right, with its own path, \$MFT entry and timestamps.
MftEntry	NTFS record number: identifies the file independently of its name.
Extracted(Utc)	When this exhibit was extracted.
SourceCreated, SourceModified, SourceMftModified, SourceAccessed	The four NTFS timestamps of the original file, in UTC and in the suspect's local time.

Why three fingerprints

MD5 collisions have been producible at will since 2008, and SHA-1 ones since 2017: a single fingerprint no longer settles an identity dispute. Three independent ones do.

5.3 Verifying a collection

5.3.1 The seal

The manifest cannot hold its own fingerprint: it is in MANIFEST.sha256. On Linux:

```
$ cd output/exhibits
$ sha256sum -c MANIFEST.sha256
MANIFEST.json: OK
```

5.3.2 Each exhibit

The `verify-exhibits.ps1` script, supplied with this manual, checks the seal, then the SHA-256 of every exhibit:

```
PS> powershell -ExecutionPolicy Bypass -File verify-exhibits.ps1 -Output E:\output
Seal verified (CEB489A143AE29EB9E3BC836F3D565B744F036AE1F3A72A3447408F31F8A5B67)
1617 exhibit(s) verified, 0 mismatch(es)
```

Its exit code is 0 if everything matches, 1 otherwise.

```
# verify-exhibits.ps1 – checks the seal and every exhibit of a WAC collection.
# Usage: powershell -ExecutionPolicy Bypass -File verify-exhibits.ps1 -Output E:\output
# Exit code: 0 if everything matches, 1 otherwise.

param([string]$Output = '.\output')

# 1. The seal: the manifest's fingerprint must be the one recorded beside it.
$recorded = ((Get-Content -Raw "$Output\exhibits\MANIFEST.sha256") -split '\s+')[0]
$actual = (Get-FileHash -Algorithm SHA256 "$Output\exhibits\MANIFEST.json").Hash
if ($recorded -ne $actual) { Write-Output "SEAL MISMATCH: the manifest was modified"; exit
    1 }
Write-Output "Seal verified ($actual)"

# 2. Every exhibit: its SHA-256 must be the one in the manifest.
$m = Get-Content -Raw -Encoding UTF8 "$Output\exhibits\MANIFEST.json" | ConvertFrom-Json
$ok = 0; $ko = 0
foreach ($p in ($m.Items | Where-Object { $_.Result -eq 'OK' })) {
    $f = Join-Path $Output $p.ExhibitPath
    $h = (Get-FileHash -Algorithm SHA256 -LiteralPath $f -ErrorAction SilentlyContinue).
    Hash
    if ($h -eq $p.SHA256) { $ok++ } else { $ko++; Write-Output "MISMATCH: $($p.ExhibitPath
    )" }
}
Write-Output "$ok exhibit(s) verified, $ko mismatch(es)"
if ($ko) { exit 1 }
```

5.4 The investigation log

`investigation.json` traces the collection: tool and command line, machine and time zones, operator, start, end and duration, then the **ordered** list of operations. For each one:

Seq, TimestampUtc, TimestampLocal
rank and time;

Operation, Target
what was done, and on what;

Result
the outcome;

Footprint

the trace this operation is expected to leave on the examined machine.

This log makes it possible to compare, collection by collection, the expected traces with those actually observed. It also flags a difference between the suspect's time zone and the collecting machine's.

5.5 The artefact files

One file per artefact. The fields described are those emitted; a field without a value is not emitted (see chapter 6).

5.5.1 System and accounts

OperatingSystem.json

machine name and domain, Windows edition and version, installation date, current time, **boot time**, uptime, time zone. `BootTimeSource` says where the boot time comes from (kernel, or fallback estimate); `ClockAdjustedSinceBootMs` gives the total clock corrections since boot — a few seconds is routine resynchronisation, a large value means a clock changed.

users.json

local accounts, read from the SAM hive: SID and RID, state, flags, logon and failure counts, dates of last logon, last failure, password change and account change, profile.

Sessions.json

sessions open at collection time: user, logon type, authentication package, start time.

processes.json

running processes: name, identifiers (and parent), owner, session, loaded modules and, with `--binary`, the executable's fingerprints.

services.json

services and drivers, configuration read from the hive: type, start mode, account, binary and hosted DLL, failure command, dependencies, last write of the key; current state; with `--binary`, fingerprints of the binary and of the DLL.

ScheduledTasks.json

scheduled tasks, read from their XML definitions: author, run-as account, state, last run and its result, triggers and actions (with `--binary`, fingerprints of the command).

5.5.2 Program execution

prefetchs.json

Prefetch files: program, run count and times, volumes, and list of loaded files with their \$MFT reference (and their fingerprints with `--binary`).

amcache_application_files.json, amcache_applications.json

Amcache inventory of executables and applications: path, publisher, version, dates.

shimcache.json

compatibility cache: path and modification date of the file. An entry proves the binary was present, not that it ran.

bams.json

last execution per program and per user (Background Activity Moderator).

userassists.json

programs launched from the user interface: count, focus time, last execution.

muicache.json

display names of executed programs.

run.json

programs started at boot or logon (Run keys), with the key's last write.

5.5.3 Files and browsing

recentdocs.json

recent-document shortcuts: target, timestamps of the shortcut and of the target, volume (type, serial number, label), network path, shell item list.

jumplistAutomaticDestinations.json, jumplistCustomDestinations.json

per-application jump lists, with the shortcuts they hold; in the automatic ones, each shortcut comes with its DestList entry (last access, machine name, tracking identifiers, pin status).

shellbags.json

folders browsed in Explorer, as a rebuilt tree.

mrus.json, mruApps.json

most-recently-used files and applications.

5.5.4 Devices

Usbstor.json

USB storage devices known to the system: Vendor, Product, Revision and Type, from DeviceId; SerialNumber, from InstanceId — absent when Windows made the identifier up (second character "&"), the device declaring none; first and last insertion dates.

mounted_device.json

mapping between drive letters and devices. Drive names the letter or the volume; Type gives the form of the value: GPT partition (PartitionGuid), MBR partition (DiskSignature and PartitionOffset, in bytes), Device path (Device, the device path, with the serial number of a USB key) or Unknown (Data, the bytes in hexadecimal).

5.5.5 Event logs (--events)

events.json holds one object per event:

EvtSystem*

the system section: provider, identifier, level, task, keywords, creation time, record number, process and thread, channel, computer, user;

EvtEventData

the event's data, as a list of Name / Value pairs;

EvtEventMessage

the readable message, as Event Viewer would show it, rebuilt from the provider's resources collected on the machine;

EvtSourceLog

the log file the event comes from. One channel can be carried by several files (current log and archives) whose record numbers overlap.

Incomplete messages

A %1, %2... mark left in a message means the event does not carry the expected data: it is kept visible so that the sentence does not look complete. Not every provider ships message resources: their events then have no EvtEventMessage, but their data are complete.

Reading the data

6.1 Timestamps

Every date is emitted twice: `X` in local time and `XUtc` in UTC. The local time is the **suspect's**, computed with the time zone read from its `SYSTEM` hive — not the collecting machine's. Dates carry the fraction of a second when the source provides it, and only to that source's precision: seven digits for a `FILETIME`, three for a date to the millisecond, none for a date to the second (FAT date, precise to two seconds, text date, Unix time).

Every date carries the offset in force **at that date**, daylight saving time included: a January date collected in summer in Paris is at `+01:00`, a July date at `+02:00`. WAC applies the rules of the examined machine year by year (Dynamic DST key of the `SOFTWARE` hive): a 2005 date in the United States follows the rules of 2005. For a value the artefact stores in local time (FAT date, Amcache, BAM...), the hour repeated in autumn is read as its first occurrence, and the hour skipped in spring with the offset before the change, as Windows does.

6.2 A missing field is not an empty field

A value with no source is not emitted at all. An empty key would read as a failed read, and could not be told apart from "the hive does not hold this value". What could not be read is recorded in `investigation.json`.

6.3 Artefact not collected

When a whole artefact could not be read, its file says so explicitly (the text is produced in French):

```
{
  "Artifact": "USBSTOR (registre)",
  "CollectionStatus": "NotCollected",
  "Error": "0x02 Le fichier spécifié est introuvable.",
  "Note": "La lecture de cet artefact a échoué : l'absence de données
          ci-dessus ne signifie PAS qu'aucune trace n'existe sur le système."
}
```

Warning

An empty artefact file and a "NotCollected" file do not say the same thing. The first means no entry was found; the second, that the source could not be read.

6.4 Undecoded data

An item WAC cannot decode (unknown shell item, unrecognised extension block) comes with its raw bytes in hexadecimal, so that an analyst can decode it later. With `--dump`, the raw value of shellbags and shortcuts is always added.

6.5 Paths

Paths and values are delivered **as they are on the disk**, untransformed: escaping happens only when the JSON is written.

Traces left on the examined machine

Read this before using WAC on a case. This chapter lists what WAC does to the machine, operation by operation, including what it cannot avoid.

7.1 What WAC still does

Operation	Trace
Direct volume reading	One access per volume actually read (usually one). No file opened, so no access date modified. If object-access auditing is enabled, opening the volume may be logged (Security 4656/4663).
Writing the results	On the collection medium only. Nothing is written to the examined disk.
Replaying hive logs	On the working copy only, with an undo journal.
Service state	One read-only query to the Service Control Manager. No service is opened or modified.
Processes	A kernel snapshot; no process is opened. Owners: one query to the Terminal Services service.
Sessions	One read-only query to LSASS.
Boot time	One in-memory query to the kernel.
Event logs (--events)	Read through the same direct volume access: the logging service is not solicited.
Referenced files (--binary)	Read through the same direct access. No file opened.
Authenticity check (--binary)	Catalogs read through the same direct access; verification in memory. No service solicited, no access to the certificate store, no network access.

7.2 What WAC no longer does

Earlier versions created a Volume Shadow Copy, went through COM and WMI, queried the Task Scheduler and the logging service, opened every process and every service, read the profile list in the live registry and opened every referenced file to fingerprint it. All these operations, which left entries in the logs or changed access dates, have been removed.

7.3 What no tool can avoid

These traces are listed so that the analyst recognises WAC's footprint instead of taking it for the activity under study:

- **running WAC.exe is an execution:** it produces a Prefetch file for WAC.exe, an Amcache entry and, if process-creation auditing is on, a Security 4688 event. Expect to find WAC in the very artefacts it collects;
- **plugging in the USB stick** registers it in USBSTOR, MountedDevices and the related logs;
- **logging on** to run the tool creates a session and its log entries;
- reading several gigabytes **evicts the file-system cache**, which changes the machine's performance state (not its disk content).

Known limitations

LZX compression

Among the Compact OS (WOF) variants, only the XPRESS ones are decompressed. A file compressed with LZX is reported as unsupported, never returned wrong.

Non-NTFS volumes

A file on a CD-ROM or a FAT stick is not read: it is reported as unread.

Per-user variables

A path written with %APPDATA%, %LOCALAPPDATA% or %USERPROFILE% (in a scheduled task, for instance) is not resolved: the account concerned is not known there, and guessing would fingerprint another file.

Deleted executable

A process whose executable has been deleted from the disk has no fingerprint: only memory is left, outside WAC's scope.

Event messages

About a third of the events get a readable message on the test machine; the other providers ship no usable resources. The data themselves are always complete.

Signed but not authenticated

A few kinds of genuinely signed files are still collected, the cautious side: Microsoft Store applications (signed at package level, not file by file) and Windows Script Host scripts signed only within their text.

Troubleshooting

Message or finding	Cause and remedy
"Check administrator rights" fails	The prompt was not started as administrator. Start it again with administrator rights.
"Check OS >= Windows 10" fails	System older than Windows 10: not supported.
"Checking the collection medium" fails, directory not empty	The output folder holds an earlier collection. Give a new folder with --output.
"Checking the collection medium" fails, not enough space	The medium is too small for the chosen options (section 3.2). Use a larger medium or drop --binary.
An artefact file holds "CollectionStatus": "NotCollected"	The source could not be read; the reason is in Error. The other artefacts are not affected.
Failed exhibits in the manifest	Referenced files missing from the disk (uninstalled software, service profile without a hive), non-NTFS volume... The reason is in Result.
The log reports binaries "hashed without being copied"	Space ran out during collection: the fingerprints are complete, the copies are not.
Need to understand a read failure	Run again with --loglevel=2 (and, for NTFS reading, --debug) and read WAC.exe.log.

Glossary

Artefact

Trace left by the activity of the system or of a user, usable in an investigation.

Exhibit store (exhibits)

Directory of the raw copies, sealed, never rewritten.

Fingerprint

Value computed on a file's content (MD5, SHA-1, SHA-256): different contents have different fingerprints.

Hive

File holding part of the registry (SYSTEM, SOFTWARE, ntuser.dat...).

Manifest

File identifying each exhibit of the exhibit store.

\$MFT

File table of an NTFS volume.

Seal

The manifest's fingerprint, kept apart: it reveals any change to the manifest.

Transaction log

.LOG1/.LOG2 file holding a hive's recent changes, not yet written into it.

Working copy (working)

Directory of the working copies, on which the analysis and any modification are made.

WOF / Compact OS

Compressed storage mode of the system files of Windows 10 and 11.